ARCHITECTURE JUSTIFICATION

Pharmacy Expiry & Re-order Dispatch Engine | Lab 3

Architecture Selection

We chose a Layered Architecture, organized into Presentation, Application/Business, and Data layers.

Two Reasons for the Choice

1. Separation of responsibilities. Pharmacy staff and suppliers use separate presentation components, while FEFO allocation, expiry monitoring, and purchase-order processing are handled by focused business components. This keeps checkout and inventory workflows understandable and makes modules easier to maintain.
2. Consistent inventory information. Stock quantities, batch validity, expiry dates, transactions, and purchase orders are stored through a centralized data layer. FEFO selection and replenishment therefore work from the same recorded inventory state, reducing inconsistent decisions.

Security Advantage

The presentation layer separates pharmacy-staff functions from supplier access. Role-based authorization can restrict checkout, batch rejection, and purchase-order dispatch to authorized staff, while suppliers can view only their own dispatched orders. Centralized database access also provides a consistent place to validate changes and maintain audit records.

Performance Benefit

Keeping FEFO allocation and inventory checks in the application layer avoids extra network hops between separately deployed services during checkout. Indexed batch and expiry fields can support fast selection of the earliest-expiring valid batch. Operational monitoring can measure the Lab 1 target that at least 95% of qualifying dispatch events complete within two seconds.

Components and Interfaces

The diagram contains eight components: Pharmacy Staff Dashboard, Supplier Portal, Inventory & Batch Management, FEFO Stock Allocation, Expiry Monitoring & Alerts, Purchase Order Management, Reporting & Performance Monitor, and Pharmacy Database. Interfaces cover stock/batch queries, FEFO allocation, eligible-batch lookup, expiry updates, minimum-stock triggers, supplier purchase-order access, dashboard reporting, and database persistence.

Trade-off

Compared with microservices, a layered design reduces deployment and network complexity for this system, but modules are less independently scalable and the centralized data tier needs backups and availability protection. It favors simpler coordination and consistent inventory handling over independent service deployment.